Project Report

Alarm Clock (Python / Tkinter)

Student Name	Pratik Arya
Registration No.	26BAI10137
Platform	VITyarthi
Language / Libraries	Python 3; tkinter, datetime, time, threading, winsound
Source File	Pratik Arya 26BAI10137 @ VITYARTHI.py

Abstract

This report describes the design and implementation of a desktop Alarm Clock built with Python and Tkinter. The user selects a time using three drop-down menus and starts the alarm with a button. A background thread compares the system time with the selected time every second and plays a WAV sound on a match. The project uses only the standard library and demonstrates GUI development, time handling and multithreading.

1. Introduction

Alarm applications are among the most common utilities. This project implements a minimal version for the desktop. Its purpose is educational as well as practical: it combines an event-driven interface (Tkinter) with a worker thread, which is a typical pattern in real applications where long-running work must not block the user interface.

2. Objectives

- Build a simple, readable GUI for setting an alarm.
- Compare selected and current time every second.
- Play a sound at the alarm time.
- Keep the GUI responsive through threading.

3. Tools and Technologies

Module	Purpose
tkinter	Window, labels, frame, OptionMenu drop-downs and button.
datetime	Reads the current system time and formats it as HH:MM:SS.
time	time.sleep(1) pauses the loop for one second between checks.
threading	Runs the alarm loop in the background to keep the window responsive.
winsound	Plays sound.wav on Windows.

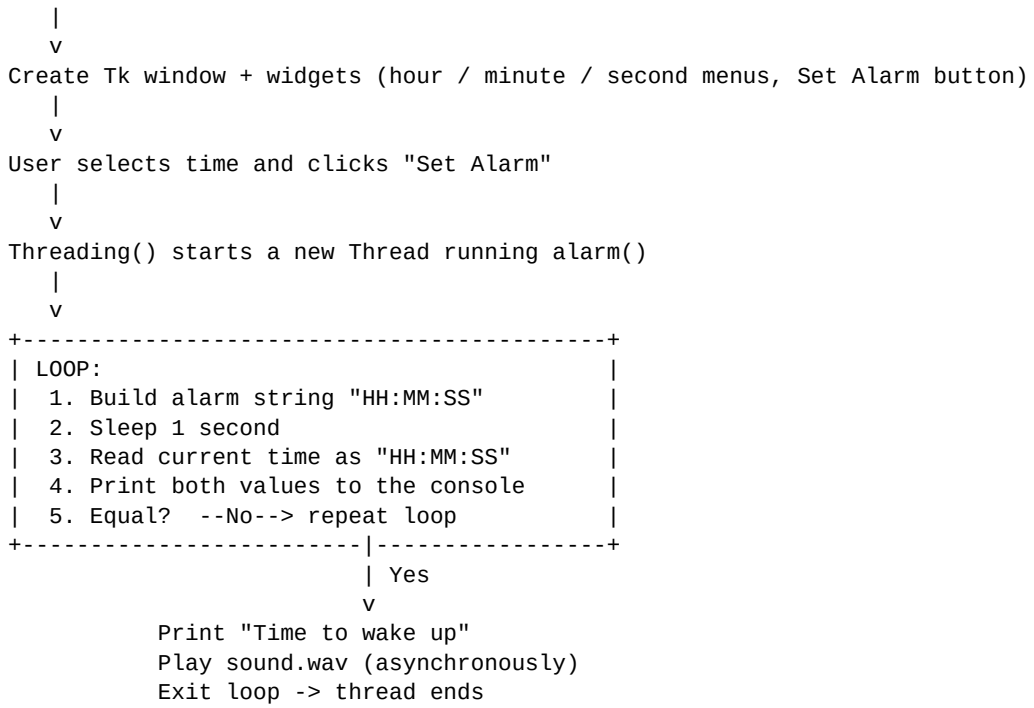
4. System Design

4.1 Functional Requirements

ID	Requirement	Implemented by
FR1	Display an application window with an 'Alarm.Clock' heading.	Tk(), Label
FR2	Choose hour, minute and second from drop-downs.	OptionMenu + StringVar
FR3	Start the alarm when 'Set Alarm' is clicked.	Button -> Threading()
FR4	Check the system time once per second.	alarm() loop, time.sleep(1)
FR5	Play a sound and stop checking on match.	winsound.PlaySound(), break

4.2 Program Flow

```
Start program
```



5. Implementation

5.1 Interface. The window is 400x200 pixels. Two red heading labels ('Alarm.Clock' and 'Set Time') sit above a frame containing three OptionMenu widgets side by side. The 'Set Alarm' button is below them.

5.2 Time selection. Three StringVar objects hold the selected hour, minute and second as two-digit strings (default '00'). Two-digit strings can be joined directly into the same HH:MM:SS format that strftime produces.

5.3 Alarm logic. The core of the program is the alarm() function:

```
def alarm():
    while True:
        set_alarm_time = f"{hour.get()}:{minute.get()}:{second.get()}"
        time.sleep(1)
        current_time = datetime.datetime.now().strftime("%H:%M:%S")
        print(current_time, set_alarm_time)
        if current_time == set_alarm_time:
            print("Time to wake up")
            winsound.PlaySound("sound.wav", winsound.SND_ASYNC)
            break
```

Each second the current time is compared with the selected time by string equality. On a match, the message is printed, the sound is played with SND_ASYNC (non-blocking) and the loop exits.

5.4 Threading. Tkinter's mainloop() must keep running to process window events, so the waiting loop cannot run on the main thread. The button calls Threading(), which creates a Thread targeting alarm() and starts it.

```
def Threading():
    t1 = Thread(target=alarm)
    t1.start()
```

6. Testing

The scenarios below describe expected behaviour and can be verified by running the program on Windows with sound.wav in the same folder.

#	Scenario	Expected result
1	Set alarm 1 minute ahead and click Set Alarm.	Console prints current and alarm time each second; at the match 'Time to wake up' appears and the sound plays.
2	Leave all menus at 00:00:00 and click Set Alarm.	Alarm fires at midnight if the program is still running.
3	Move/resize the window while waiting.	Window stays responsive (loop is on a separate thread).
4	Select a time earlier than the current time.	Alarm does not fire until that time comes around next day.
5	Run without sound.wav.	Message printed; Windows may play a default sound or raise an error.

7. Results and Discussion

The program meets its core objectives: the user can choose a time, the application checks the clock every second, and a sound is played at the selected time while the interface remains responsive. The use of a separate thread is the key design decision that keeps the window from freezing. The code is short and readable, which makes it a good base for enhancement.

8. Limitations

- **Windows only** - winsound is unavailable on macOS/Linux.
- **Invalid options:** the hour menu includes '24' and minute/second menus include '60'; alarms set with these never ring.
- **No cancel option;** pressing Set Alarm repeatedly starts multiple threads.
- **Non-daemon thread:** closing the window while waiting may leave the program running until the alarm fires.
- **Exact-match check:** a delayed iteration that skips a second can miss the alarm.
- **No on-screen feedback;** status appears only in the console.
- Requires sound.wav in the working directory.

9. Future Enhancements

- Limit menus to valid values (00-23, 00-59) or use Spinbox widgets.
- Add Stop / Snooze buttons using threading.Event; use daemon threads.
- Compare with >= so an alarm is never missed.
- Show a live clock and 'Alarm set for HH:MM:SS' status in the window.
- Support multiple alarms, repeat days and custom ringtones.
- Use a cross-platform audio library (playsound, pygame).

10. Conclusion

The project delivers a working desktop alarm clock in under 70 lines of Python using only the standard library. It demonstrates GUI programming with Tkinter, use of system time and multithreading. With input validation, cancel/snooze support and cross-platform sound, it could grow into a more complete application.

References

- Python Documentation - tkinter: https://docs.python.org/3/library/tkinter.html
- Python Documentation - threading: https://docs.python.org/3/library/threading.html
- Python Documentation - datetime: https://docs.python.org/3/library/datetime.html
- Python Documentation - winsound: https://docs.python.org/3/library/winsound.html